

4 – REPETITION

Repetition structures allow a block of code to run zero or more times. Without loops, many practical programs would be inefficient or outright impractical – for example, adding the numbers from one to one million. Loops are essential for processing collections, performing repeated calculations and automating repetitive tasks.

Python supports two main looping constructs:

- `while` loops
- `for` loops

Python does **not** support:

- do-while loops
- C-style `for(init; condition; update)` loops

Both loop types support the `break` and `continue` keywords.

4.a – While statements

The `while` loop performs **conditional iteration**. The statements in the loop body are processed **as long as** the condition is `True` and processing ceases when the assertion becomes `False`. It is useful when the number of iterations is not known in advance.

```
while «condition»:  
... # body of the loop
```

If the condition never becomes `False`, you will have created an infinite loop. Always ensure your loop has logic to eventually terminate.

```
>>> x = 1  
>>> total = 0  
>>> while x < 100:  
...     total += x  
...     x += 1  
...  
>>> print(total)
```

4950

4.b – For loop

A for loop iterates over the elements of an iterable, so the number of iterations matches the number of elements produced.

Common Iterables:

- strings
- lists
- tuples
- sets
- dictionaries
- range() object

4.b.1 – Iterating Over an Iterable

Iterables include strings, lists, sets, tuples and dictionaries.

```
# list example
>>>a = [3, 5, 7, 9]
>>>for x in a:
...   print(x, end=' ')
...
3 5 7 9 >>>
```

```
# string example
>>> name = 'Narendra'
>>> for a in name:
...   print(a, end=' ')
...
N a r e n d r a >>>
```

```
fruits = ['apple', 'banana', 'orange']
for f in fruits:          #fruits is a list
    print(f)
```

4.b.2 – Using the range object

The `range()` function returns an immutable sequence of numbers. A for loop is normally used to iterate this sequence. The range function takes one mandatory and two optional arguments.

```
range(stop)
range(start, stop)
range(start, stop, step)
```

```
>>>for x in range(3, 10, 2):
...   print(x, end=', ')
...
3, 5, 7, 9 >>>
```

4.b.3 – Using the `_` (Discard Variable)

If the loop variable is not needed, use the `_`. This avoids linter warning and clearly signals intent.

```
# if you are not using the value, then the linter
# will complain about un-used variables. The under
# score (referred to as a 'discard') prevents this
# often annoying reporting.
>>>for _ in range( 10 ):
```

```
... print('Hello', end=' ')\nHello Hello Hello Hello Hello Hello Hello Hello Hello Hello
```

4.b.4 – Using the Enumerate Function

The `enumerate()` function returns index-value pairs from a collection (a sequence or mapping).

```
enumerate(«collection», [«start=0»])
```

```
>>> profs='ilia yin arben narendra mehrdad'.split()\n>>> #process all the items\n... for index, item in enumerate(profs):\n...     print(f'{index} {item}')\n...\n0 ilia\n1 yin\n2 arben\n3 narendra\n4 mehrdad
```

```
# you can start the count at a particular integer\n>>> profs='ilia yin arben narendra mehrdad'.split()\n>>> for index, item in enumerate(profs, start=2):\n...     print(f'{index} {item}')\n...\n2 ilia\n3 yin\n4 arben\n5 narendra
```

6 mehrdad

```
# you can iterate a dictionary
>>> profs = {'ilia': 'COMP304', 'yin': 'COMP306', 'arben': 'COMP305',
            'narendra': 'COMP216', 'mehrdad': 'COMP225'}
>>> for index, key in enumerate(profs):
...     print(f'{index}: {key}')
...
0: ilia
1: yin
2: arben
3: narendra
4: mehrdad
```

```
# a better way to iterate a dictionary is to use the .items() method of the
dictionary class
>>>
>>> for prof, course in profs.items():
...     print(f'{prof.rjust(12)}: {course}')
...
      ilia: COMP304
      yin: COMP306
      arben: COMP305
narendra: COMP216
mehrdad: COMP225
```

4.b.5 – Nested Loop

A loop within another loop is referred to as a nested loop. The two loops (inner and outer) do not need to match. It is possible to have multiple-level of nested loops.

```
distros = 'ubuntu mint centos'.split()
arch = '32-bit 64-bit'.split()
for d in distros:
    for a in arch:
        print(d, a)
    print('_____')
```

```
ubuntu 32-bit
ubuntu 64-bit
```

```
_____
mint 32-bit
mint 64-bit
```

```
_____
centos 32-bit
centos 64-bit
```

4.c – Loop control Keywords

4.c.1 – break

This keyword allows you to break free of the loop. You can use it in a `for` or `while` loop to terminate when a particular condition is detected.

```
scores = [34, 67, 99, 105]
for s in scores:
    if s > 100:
        print('Invalid')
        break
    print(s)
```

4.c.2 – continue

This keyword allows you to skip the rest of the statement in the loop body and go to the next. The loop will still continue, but it will start from the top of the loop statement.

```
n = 0
while n < 100:
    n += 1
    if n % 2 == 0:
        continue
    print(n)
#Prints all the odd numbers
```

4.c.3 – pass

This keyword allows you to place a nothing statement (one that does not do anything) where a statement is needed. It is often used when you are not sure or you have not decided on the implementation details. It can be used for the body of a class, a method, the body of a loop or a block in an if statement.

```
for x in range(20):
    if x % 3 == 0:
        pass          #a nothing statement
    else:
        print(x)
```

4.d – Infinite Loops

Sometimes, the condition you are checking to see if the loop should continue never becomes false. In this case, the loop will continue for eternity (or until your computer stops it). This is more common with while loops.

```
# you may stop the following loop by pressing ctrl+c
while 5 > 1:
    print("I'm a survivor")
    input('')
```

4.e – Summary

This chapter introduced repetition structures, which allow a block of code to execute zero or more times. Loops are essential for processing collections, automating repetitive tasks, and writing efficient programs — without them, tasks like summing a million numbers would be impractical.

Key takeaways:

- Two loop constructs in Python:
- `while` — conditional iteration; repeats as long as a condition is `True`. Best when the number of iterations is not known in advance.
- `for` — determinate iteration; iterates over the elements of an iterable (strings, lists, tuples, sets, dictionaries, `range()` objects, etc.).
- Python does not support do-while loops or C-style `for(init; condition; update)` loops.
- `range()` generates an immutable sequence of numbers. It supports one, two, or three arguments: `range(stop)`, `range(start, stop)`, and `range(start, stop, step)`.
- `enumerate()` yields index-value pairs, useful for tracking position while iterating. It accepts an optional start argument.
- `zip()` (not covered in the original but worth knowing) iterates multiple iterables in parallel.
- **Nested loops** place one loop inside another; the inner loop runs completely for each iteration of the outer loop.
- **Loop control keywords:**
 - `break` — exits the loop immediately.
 - `continue` — skips the rest of the current iteration and moves to the next.
 - `pass` — a no-op placeholder, used when a statement is syntactically required but no action is needed yet.
- **Infinite loops** occur when the loop condition never becomes `False`. Always ensure termination logic; use `Ctrl+C` to interrupt a runaway loop. A common intentional pattern is `while True:` with a `break` inside.
- Loop else clause (a Python-specific feature): the `else` block after a `for` or `while` runs only if the loop finished without hitting a `break`.

In short: while loops are condition-driven, for loops are collection-driven, and the combination of `break`, `continue`, and `pass` gives fine-grained control over loop behavior. Proper termination is critical to avoid infinite loops.

4.f – Non coding exercises

1. Which loop is best when the number of iterations is not known in advance?

2. What does `range(2, 10, 3)` produce?
3. What does `enumerate(['a', 'b'])` yield?
4. Which keyword exits a loop immediately?
5. Which keyword does nothing and is used as a placeholder?
6. How many arguments can `range()` accept?
7. What does the `else` clause on a `for` loop do?
8. Python supports `do-while` loops. (True or false?)
9. A `while` loop always runs at least once. (True or false?)
10. `break` and `continue` can be used in both `for` and `while` loops. (True or false?)
11. `range(5)` produces the numbers 1 through 5. (True or false?)
12. `enumerate()` can start counting at a custom number. (True or false?)
13. A nested loop's inner loop runs once per outer-loop iteration. (True or false?)
14. `pass` terminates the loop. (True or false?)
15. What the difference between a `while` loop and a `for` loop. Give a scenario where each is the better choice.
16. Why does Python use `_` as a loop variable? What problem does it solve?
17. What is an infinite loop? How can you avoid one? How can you stop one that is already running?
18. When would you use `enumerate()` instead of a plain `for` loop?
19. What is a nested loop? Give a real-world example where nesting is natural.
20. Why might you use `while True:` with a `break` instead of a normal `while` condition?
21. Explain the difference between `break`, `continue`, and `pass`.